PrepStack

// FREE CHEAT SHEET · EDITION 1

# GIT CHEAT SHEET

*for absolute beginners*


01 What Git does, and the 4 places your code lives

02 The 5 commands for your first week

03 A first-week workflow, step by step

04 More commands you will need soon

05 Common errors and how to fix them

06 Five beginner Git interview questions

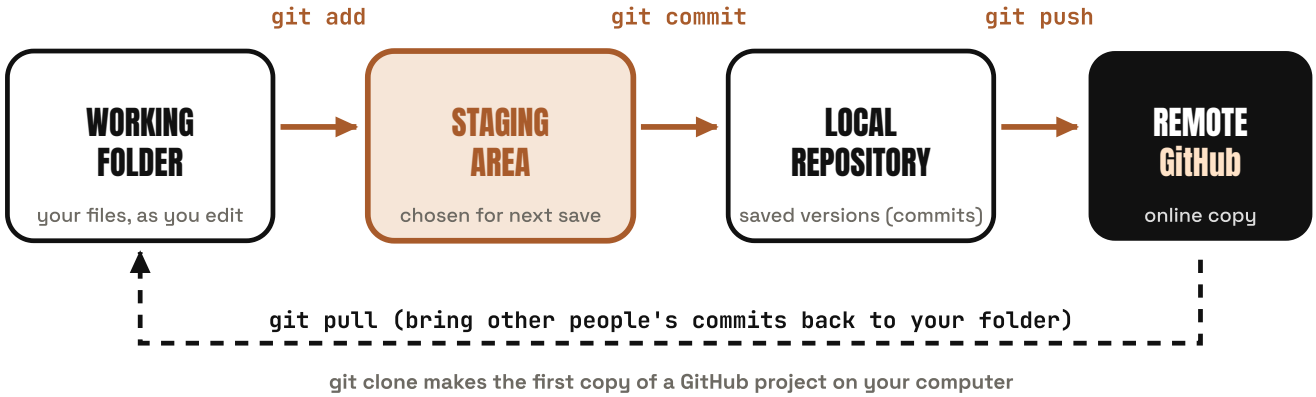

**LEARN. BUILD. GET HIRED.**

Follow **@prep_stackk** for more tech concepts made simple

# GIT IN *one minute*

**Git** is a tool that runs on your computer and saves **versions** of your code, so you can see what changed, go back when something breaks, and work with other people without overwriting each other. **GitHub** is a separate website that stores Git projects online.

## THE 4 PLACES YOUR CODE LIVES




git clone makes the first copy of a GitHub project on your computer

> **Remember this order:** edit files, **add** the ones you want, **commit** them with a message, **push** to GitHub. **Pull** before you start, and before you push.

## WHAT IS INSIDE

01 The 5 commands for your first week p. 3

02 A first-week workflow, step by step p. 4

03 More commands you will need soon p. 5

04 Common errors and how to fix them p. 6

05 Five beginner Git interview questions p. 7-8

Commands are shown for the terminal (Git Bash on Windows). The default branch name is usually `main`; some older setups use `master`. All examples use made-up names.

# THE 5 COMMANDS FOR *week one*

Learn these five first. They cover most of what you do in your first weeks with Git.

01 · START

`git clone <url>`

Copies a project from GitHub to your computer, with its whole history. Starting a brand-new project instead? Run `git init` inside its folder.

02 · LOOK

`git status`

Shows which files changed, which are staged for the next commit and which are new. Run it before every `add` and `commit`. It is the command that tells you what is going on.

03 · CHOOSE

`git add <file>` or `git add .`

Puts changes in the staging area, which is the list of what goes into the next commit. `.` means "everything that changed here". Nothing staged means nothing gets committed.

04 · SAVE

`git commit -m "message"`

Saves a snapshot of the staged changes in your local history. Write the message like a note to your future self: what changed and why, for example `"Add login form validation"`.

05 · SHARE

`git push`

Sends your new commits to GitHub. The first time on a new branch, use `git push -u origin main` (it remembers where to push). If it says "rejected", run `git pull` first.

**Bonus pair:** `git pull` brings other people's commits to your computer, and `git log --oneline` shows the history in one line per commit. Both are on page 5.

# A FIRST-WEEK *workflow*

One full round, from copying a project to getting your change on GitHub. The output lines are examples; yours will differ slightly.

1. **Copy the project once.**

```
git clone https://github.com/your-name/my-project.git
cd my-project
```

2. **Start every session by getting the latest.**

```
git pull
```

3. **Edit a file** (here `index.html`), **then check what Git sees.**

```
git status
# Changes not staged for commit:
#         modified:   index.html
```

4. **Stage it, check, then commit with a clear message.**

```
git add index.html
git status
# Changes to be committed:
#         modified:   index.html
git commit -m "Add contact form to the home page"
# [main 3f2a9c1] Add contact form to the home page
```

5. **Send it to GitHub.**

```
git push
# main → main
```

**Small commits win.** One idea per commit makes history readable and mistakes easy to undo.

**Pull before you push.** If someone pushed first, Git rejects your push until you `git pull`.

# MORE COMMANDS *you will need*

| Command | What it does |
|---|---|
| `git pull` | Fetches other people's commits from GitHub and merges them into your current branch. |
| `git log --oneline` | Shows the history, one line per commit (short id + message). |
| `git diff` | Shows exactly which lines you changed (unstaged changes). `git diff --staged` shows the staged ones. |
| `git branch` | Lists branches. A branch is a separate line of work. |
| `git switch -c new-feature` | Creates a new branch and moves to it. `git switch main` goes back. |
| `git merge new-feature` | Brings the commits of `new-feature` into the branch you are on. |
| `git restore --staged <file>` | Un-stages a file (the change stays in your folder). |
| `git restore <file>` | **Discards** your unsaved changes to that file. It cannot be undone, so check `git status` first. |

## .GITIGNORE: FILES GIT MUST NOT TRACK

Create a file named `.gitignore` in the project folder and list names or patterns, one per line:

```
.env              # passwords and API keys never go to GitHub
node_modules/     # downloaded packages, can be reinstalled
*.log             # log files
```

> **Committed a secret by mistake?** Deleting the file in a new commit does not remove it from the history. Treat the key or password as leaked: revoke it and create a new one.

## SET UP GIT ONCE

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Git puts this name and email on every commit you make. Use the email you want to be public on GitHub (GitHub offers a private "noreply" address in its settings).

# COMMON ERRORS *and fixes*

You will meet these in week one. Read the message, because Git usually tells you the next step.

| What you see | What it means | Fix |
|---|---|---|
| `fatal: not a git repository` | You are in a folder that Git does not manage. | `cd` into the project folder, or run `git init` for a new project. |
| `nothing to commit, working tree clean` | No changes since the last commit. | Edit a file first. Nothing to fix. |
| `no changes added to commit` | You changed files but did not stage them. | `git add <file>`, then commit. |
| `! [rejected] ... (fetch first)` | GitHub has commits you do not have. | `git pull`, then `git push`. |
| `Please tell me who you are` | Git does not know your name and email yet. | Run the two `git config --global` lines (page 5). |
| `Authentication failed / Permission denied` | GitHub did not accept your login. It no longer accepts your account password for Git over HTTPS. | Sign in through Git Credential Manager or GitHub Desktop, use a personal access token, or set up an SSH key. |
| `CONFLICT (content): Merge conflict in index.html` | You and someone else changed the same lines. | See below. |

## FIXING A MERGE CONFLICT, STEP BY STEP

1. Open the file. Git marks the clash:

```
<<<<<<< HEAD
your version of the line
=======
their version of the line
>>>>>>> origin/main
```

2. Edit the file so it has the final text you want, and **delete the three marker lines.**
3. `git add index.html`, then `git commit` to finish the merge.

> **Not scary:** a conflict only means Git needs a human to choose. Pulling often keeps conflicts small.

# 5 GIT INTERVIEW *questions*

Beginner-level questions that come up in software interviews. Learn the idea, then answer in your own words.

### Q1 What is the difference between Git and GitHub?

**ANSWER** **Git** is a version control tool that runs on your computer and tracks changes to your code. **GitHub** is a website that hosts Git repositories online, so you can back them up, share them and collaborate (pull requests, code review). You can use Git without GitHub, but GitHub is built around Git.

**AVOID** Saying they are the same thing, or that GitHub is "the Git app". Other hosts, such as GitLab and Bitbucket, also use Git.

### Q2 What is the difference between `git add` and `git commit`?

**ANSWER** `git add` moves changes into the **staging area**, which is the list of what will be in the next commit. `git commit` takes everything staged and saves it as a snapshot in the local history. The staging step lets you commit only some of your changes, with a message for each logical piece.

**FOLLOW-UP** "Does commit send code to GitHub?" No. A commit is local. `git push` sends it.

### Q3 What is the difference between `git fetch` and `git pull`?

**ANSWER** `git fetch` downloads new commits from the remote but does not change your working files. `git pull` is a fetch followed by a merge into your current branch, so it updates your files, and it can cause merge conflicts.

**WHEN** Use fetch to look at what changed before you merge. Use pull when you just want to catch up.

### Q4 What is a merge conflict, and how do you resolve it?

**ANSWER** A conflict happens when two branches change the same lines (or one deletes a file the other edits) and Git cannot decide which to keep. Git marks the file with `<<<<<<<`, `=======` and `>>>>>>>`. You edit the file to the final version, remove the markers, `git add` it and commit.

### Q5 What is the difference between `git revert` and `git reset`?

**ANSWER** `git revert <commit>` creates a **new commit** that undoes an earlier one, so the history stays intact. This is the safe choice for commits you have already pushed. `git reset` moves the branch back to an earlier commit and can rewrite history, so avoid it on commits others may already have.

**FOLLOW-UP** "What is a branch?" A lightweight, movable pointer to a commit. It lets you work on a feature without touching `main`.

## CHEAT SHEET

| I want to... | Command |
|---|---|
| copy a project from GitHub | `git clone <url>` |
| see what changed | `git status` / `git diff` |
| stage my changes | `git add <file>` / `git add .` |
| save a snapshot | `git commit -m "message"` |
| send it to GitHub | `git push` |
| get other people's changes | `git pull` |
| see the history | `git log --oneline` |
| start a branch | `git switch -c <name>` |

**Found this useful?** Follow **@prep_stackk** for more tech concepts made simple.
Learn. Build. Get Hired.